Cryptography with PPL

Sunkwan Kim - CS 4973 Project Proposal

1 Team Composition

I (Sunkwan) will be working individually on this project. Currently, I am planning to apply to master's programs with a thesis option, so I want to use this opportunity to create a personal project to aid my application process.

2 Motivation and Problem Statement

My area of interest is cryptography, and the class demo on using Roulette to perform cryptanalysis on the Caesar cipher gave me an idea to use Roulette (or perhaps a custom probabilistic PL) to analyze various cryptographic systems. In cryptography, we often deal with probabilities, which can be hard to conceptualize. In particular, we rely on several hardness assumptions, and when advantage probabilities carry over from other hard problems, it could be difficult to verify the inequalities in your head. This difficulty motivates us to use a PPL to first demonstrate that adversaries' advantages are indeed negligible in security games and to create a programmed model to perhaps modularize each system or primitive for future cryptanalysis. More directly:
Hypothesis: A Probabilistic Programming Language can be used to model cryptographic proofs and show the relationship between related schemes (e.g., an adversary's advantage of some block-cipher schemes is dependent on the security of a PRF).
Research Question: To what extent can the Probabilistic Programming Language be used to model cryptographic security games and schemes and to show their compound adversary advantages?
Evidence: Explained in *Section 4 Measures of Success*

3 Course Relevance

This project is relevant to the CS 4973 course in several ways. First, it demonstrates my familiarity with probabilistic programming languages by translating several cryptographic schemes into probabilistic programs. I thought modeling cryptographic schemes alone was insufficient for course relevance; I decided that developing a cryptography-tailored PPL would show my understanding of PPL syntax and semantics. Finally, cryptography relies heavily on randomness and probabilistic reasoning, so this project will deepen my probabilistic reasoning.

4 Measures of Success

The ideal outcome of this project is to successfully develop a programming language suited to probabilistic cryptographic analysis and create models for various security games and

schemes. More specifically, the project should be assessed according to the evaluation methodology below:

- **Correctness** for Probabilistic Programming Language *(baseline - Roulette)*
  - Evaluate general probability features such as `flip` and `let` and compare the results with a more-established language like `Roulette`. Unless a specific design choice is provided, the results should exactly match those of Roulette.
- **Correctness** for Security Games/Scheme Modeling *(baseline - Textbooks)*
  - For each model, evaluate both secure and insecure schemes (from textbooks), and the model should clearly confirm that their advantages differ. For secure schemes, the adversary's advantage should be "close" to zero; for insecure schemes, it should be "close" to 1. Hence, the models should be able to detect both security and insecurity of schemes.
  - For each model, evaluate its advantages and compare them with textbook advantages. For given n inputs, the probabilities should exactly match the results in the textbook.
- **Level of difficulty** for Security Games/Scheme Modeling ("To what extent" part of the research question)
  - Schemes will be modeled in the order of increasing difficulty. Here, "increasing difficulty" would mean the required dependence on other schemes (or other foundational building blocks). For example, some block ciphers depend on PRFs, so block ciphers will be more difficult to model than PRFs (which is my assumption).

5 Approach

Since this project has two parts, I will simultaneously create a custom language and build models. For the language, I plan to wait for the class content to build up so that I can use as much material as possible. Meanwhile, I plan to prototype the essential models in Roulette. To start off, I have created a Semantic Security Game model to test the OTP in Roulette (shown in the tables below), and I plan to create other models in a similar manner. The following diagram and definition of the semantic security game are taken from *A Graduate Course in Applied Cryptography* textbook by Dan Boneh and Victor Shoup.

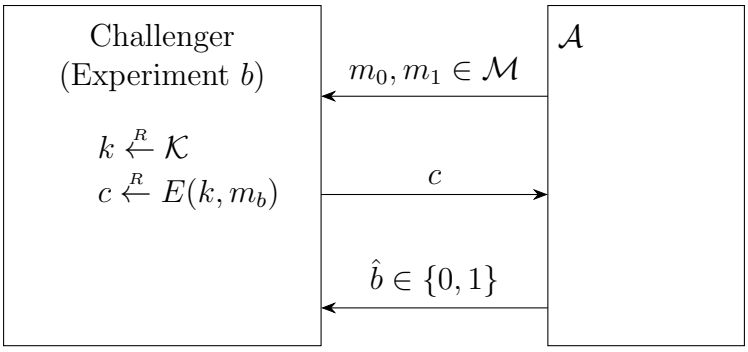


Figure 1: Experiment b of the semantic security attack game. Redrawn from Boneh and Shoup [1], Figure 2.1, p. 16.

Secure Cipher: One-Time Pad

```
(define cipher-bitwise
  (hash
   'encrypt
   (lambda (k m)
   (bitwise-xor k m))
   'decrypt
   (lambda (k c)
   (bitwise-xor k c))))
```

Insecure Cipher: Identity Cipher

```
(define cipher-identity
  (hash
   'encrypt
   (lambda (k m) m)
   'decrypt
   (lambda (k c) c)))
```

Random Guess Adversary

```
(define adversary-rand-guess
  (hash
   'choose
   (lambda (l)
   (values (sample-bits l)
   (sample-bits l)))
   'guess
   (lambda (c)
   (sample-bit))))
```

MSB Guess Adversary

```
(define adversary-first-bit
  (hash
   'choose
   (lambda (l)
   (values (make-list l 0)
   (make-list l 1)))
   'guess
   (lambda (c)
   (if (equal? (car c) 0) 0 1))))
```

Semantic Security Attack Game

```
(define (attack-game b cipher adversary l)
  (define-values (m0 m1)
    ((hash-ref adversary 'choose) l))
  (define k
    (sample-bits l))
  (define c
    ((hash-ref cipher 'encrypt) k (if (equal? b 0) m0 m1)))
  (define b-guess
    ((hash-ref adversary 'guess) c))
  (equal? b-guess 1))
```

6 Project Milestones

- **Milestone 1**: Create a new probabilistic `#lang` specific to cryptanalysis
- **Milestone 2**: Model various security games (e.g., Chosen-Plaintext Attack, Chosen-Ciphertext Attack, Authenticated Encryption, etc.) where we can put in different schemes as inputs
- **Milestone 3 (Stretch)**: Model PRF-based Encryptions & Interactive Proofs

7 Risks and Mitigation

- Milestone 1 - **Low Risk**: I considered creating a new `#lang` to be low risk since I am planning to not deviate much from the Roulette program. Additionally, I can use the course resources the most in this milestone. To further mitigate this risk, I plan to make the initial models using Roulette.
- Milestone 2 - **Medium Risk**: This milestone is the crux of this project, and it is set at medium risk since some of the schemes could be difficult to implement in Roulette/custom language. To mitigate this risk, I will start with easier models and try to make the models generalizable.
- Milestone 3 - **High Risk**: These models are higher "level of difficulty" as I explained in the Section 4. Some of these models might require modeling Number Theoretical concepts such as sampling from a Finite Group, which could be higher risk when implementing. To mitigate this risk, I will research existing papers and projects and try to translate them into my PPL.

8 Detailed Timeline

- **Week 1:** Get basic project filesystem, reader, and expander set up. Research existing cryptanalysis languages and tools. Research possible security-games/schemes/proofs candidates to model.
- **Week 2:** Implement probabilistic features in the language (similar to Roulette). Implement cryptography-specific features in the language (e.g., n-bit sampling, sampling from finite groups). Model basic security games such as IND-CPA and IND-CCA.
- **Week 3:** Model additional schemes researched from week 1.
- **Week 4:** Model additional schemes researched from week 1. Finish *Milestone Report*.
- **Week 5:** Model PRF-based Encryptions and other harder schemes.
- **Week 6:** Model Interactive Proofs related schemes.
- **Week 7:** Model Interactive Proofs related schemes. Finish *Project Report*.

References

[1] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. Version 0.6, January 2023. Available at `https://toc.cryptobook.us/`.